SKETCH2FACE

Forensic Image Synthesis System

AI-Powered Sketch-to-Face Synthesis using Pix2Pix GAN & CodeFormer Transformers

Authors	M. Ahmed Moazzam Sharif | Bilal Asif | Muaz Ahmed
Institution	NASTP Institute of Information Technology
Course	AI335L — Deep Learning Lab
Academic Year	2025 / 2026
Report Type	Capstone Project Final Report

Table of Contents

1. Problem Definition

Forensic facial reconstruction is a critical component of criminal investigation workflows. Law enforcement agencies routinely rely on witness-provided facial descriptions to generate suspect portraits for database matching, public alerts, and investigation continuity. Traditionally, this process depends entirely on the skill of a trained forensic sketch artist working directly with a witness — a method inherently constrained by subjectivity, human cognitive limitations, and time pressure.

The fundamental challenge lies in the semantic gap between a hand-drawn sketch and a photorealistic face image. Sketches produced by different artists for the same subject vary dramatically in line weight, style, shading, and abstraction level. Even professionally produced composite images generated using digital tools (FACES, IdentiKit, E-FIT) retain significant stylistic artifacts that hinder automated facial recognition systems.

1.1 Core Research Problems

Subjective Sketch Generation: Witness recall is imperfect and highly variable. Two witnesses describing the same face may produce sketches with significantly different structural representations, making standardized automated processing extremely difficult.

Identity Preservation: Standard image synthesis networks (style transfer, generic autoencoders) excel at generating aesthetically pleasing faces but systematically fail to preserve the fine-grained spatial relationships that define individual identity — eye socket depth, nasal bridge width, chin profile, and ear morphology.

Absence of Quantitative Verification: Existing forensic sketch synthesis systems provide no mathematical measure of how faithfully the output portrait corresponds to the input sketch. Human review introduces subjectivity and cannot serve as reliable evidentiary documentation.

Computational Inaccessibility: Prior academic approaches to sketch-to-face synthesis required highly specialized hardware and complex multi-stage manual pipelines inaccessible to frontline law enforcement personnel without deep technical expertise.

1.2 Proposed Solution

Sketch2Face addresses all four identified problems through a unified, end-to-end deep learning pipeline combining three architecturally distinct subsystems:

- Pix2Pix Conditional GAN (U-Net 256 + PatchGAN) — translates sketch geometry to realistic facial texture
- CodeFormer Blind Face Restoration (VQ-GAN + Transformer) — eliminates artifacts and restores fine detail
- SSIM Consistency Verification — provides a quantitative identity-preservation score (0–100%)

> **Key Objective: Produce a photorealistic, identity-faithful portrait from a hand-drawn suspect sketch with a quantifiable reconstruction confidence score, deployable in a real-time web interface accessible to non-technical forensic personnel.**

2. Dataset Description

The training corpus for Sketch2Face was derived from the CelebA-HQ (Celebrity Faces Attributes High Quality) dataset, selected for its large-scale, high-resolution, and demographically diverse collection of frontal facial photographs.

2.1 Source Dataset — CelebA-HQ

Attribute	Value
Original Source	CelebA-HQ (Large-Scale CelebFaces Attributes Dataset)
Original Paper	Liu et al., 2015 — ICCV (Deep Learning Face Attributes in the Wild)
Base Image Count	202,599 celebrity face photographs
Resolution Used	256 × 256 pixels (centre-cropped square)
Color Space	RGB
Training Pairs Constructed	28,000 paired (sketch, face) samples
Train / Val / Test Split	24,000 / 2,000 / 2,000
Subject Diversity	Multiple ethnicities, age groups, and genders

Table 2.1 — CelebA-HQ Dataset Summary

2.2 Paired Sample Construction

Raw CelebA-HQ photographs were not directly usable as training data since Sketch2Face requires pixel-aligned (sketch, face) pairs. Each pair was programmatically constructed using the following pipeline:

- Face Detection & Cropping — OpenCV Haar Cascade with 40% padding extracts a normalized frontal face crop at 256×256.
- Lineart Sketch Extraction — ControlNet LineartDetector converts the cropped photograph into a clean contour sketch representation, capturing facial structure without photographic texture.
- Pair Concatenation — The sketch and original face crop are concatenated side-by-side into a 512×256 paired image as required by the Pix2Pix training format.
- Quality Filtering — Pairs with missing face detections or degraded lineart (> 5% empty pixels) were discarded, ensuring dataset consistency.

> **Dataset Insight: The ControlNet Lineart extractor was chosen over traditional edge detectors (Canny, Sobel) because it produces semantically aware outlines that capture facial structures (eyelids, lips, nasal bridge) rather than arbitrary high-frequency edges, significantly improving GAN training stability.**

3. Preprocessing Pipeline

All input images — whether training photographs or live user uploads — pass through a standardized preprocessing pipeline before entering the synthesis network. This ensures spatial normalization, consistent sketch quality, and format compatibility across both training and inference modes.

3.1 Face Detection & Cropping

OpenCV's Haar Cascade classifier (haarcascade_frontalface_default.xml) detects the largest frontal face region in the input image. A configurable padding factor (default 40%) extends the bounding box to include hair, ears, and chin, then the region is centre-cropped to a square and resized to 256×256 pixels.

```
def crop_face(bgr_image, padding=0.4):
    gray    = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2GRAY)
    cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
    faces   = cascade.detectMultiScale(gray, scaleFactor=1.1,
                                       minNeighbors=5, minSize=(80, 80))
    if len(faces) == 0:
        h, w = bgr_image.shape[:2]
        s = min(h, w)
        return bgr_image[(h-s)//2:(h-s)//2+s, (w-s)//2:(w-s)//2+s]
    x, y, w, h = sorted(faces, key=lambda f: f[2]*f[3], reverse=True)[0]
    pad = int(max(w, h) * padding)
    x1, y1 = max(0, x - pad), max(0, y - pad)
    x2, y2 = min(bgr_image.shape[1], x+w+pad), min(bgr_image.shape[0], y+h+pad)
    crop = bgr_image[y1:y2, x1:x2]
    s    = min(crop.shape[:2])
    cx, cy = crop.shape[1]//2, crop.shape[0]//2
    return crop[cy-s//2:cy+s//2, cx-s//2:cx+s//2]
```

Code 3.1 — Face detection and square crop extraction

3.2 Lineart Sketch Extraction

Cropped face images are converted into contour sketches using the ControlNet Annotators LineartDetector, a deep learning-based edge detector trained to produce clean, semantically meaningful facial outlines:

```
from controlnet_aux import LineartDetector
from PIL import Image, ImageOps

@st.cache_resource
def load_detector():
    return LineartDetector.from_pretrained('lllyasviel/Annotators')

def make_sketch(bgr_256):
    detector = load_detector()
    pil      = Image.fromarray(cv2.cvtColor(bgr_256, cv2.COLOR_BGR2RGB))
    lineart  = detector(pil, detect_resolution=256, image_resolution=256)
    lineart  = ImageOps.invert(lineart.convert('RGB'))
    return cv2.cvtColor(np.array(lineart), cv2.COLOR_RGB2BGR)
```

Code 3.2 — Lineart sketch extraction using ControlNet Annotators

3.3 Pix2Pix Input Formatting

```
def save_pix2pix_input(sketch_bgr, save_path):
    sketch_rgb = cv2.cvtColor(sketch_bgr, cv2.COLOR_BGR2RGB)
    blank      = np.zeros_like(sketch_rgb)
    paired     = np.concatenate([sketch_rgb, blank], axis=1)    # 512x256
```

```
Image.fromarray(paired).save(save_path)
```

Code 3.3 — Constructing Pix2Pix paired input format

3.4 Preprocessing Summary

Step	Method	Output
Face Detection	Haar Cascade (OpenCV)	Bounding box coordinates
Face Crop	Padded square crop → resize	256×256 BGR image
Sketch Extraction	ControlNet LineartDetector	256×256 inverted lineart
Format Conversion	Concatenation + zeros fill	512×256 Pix2Pix input pair
Normalization	Built into Pix2Pix test.py transforms	Tensor in [-1, 1]

Table 3.1 — Preprocessing pipeline summary

4. System Architecture & Pseudocode

The Sketch2Face system comprises five architectural components operating sequentially. Each subsystem is described below with a layer-by-layer block diagram and accompanying pseudocode.

4.1 Haar Cascade Face Detector

```
┌──────────────────────────────────────────────────────────┐
│            HAAR CASCADE FACE DETECTOR — BLOCK DIAGRAM    │
├──────────────────────────────────────────────────────────┤
│  INPUT: BGR Image (arbitrary resolution)                 │
│                                                          │
│  [1] Convert to Grayscale  (cv2.cvtColor)                │
│  [2] Integral Image Computation                          │
│       Summed area table for rapid Haar feature evaluation│
│  [3] Sliding Window + Scale Pyramid                      │
│       scaleFactor=1.1 | minNeighbors=5 | minSize=(80,80) │
│  [4] Cascade of Classifiers (38 stages)                  │
│       Each stage eliminates non-face regions early       │
│  [5] Non-Maximum Suppression                             │
│       Groups overlapping detection windows → final bbox  │
│  [6] Largest Face Selection + Padded Crop (padding=0.4)  │
│       Centre-square crop → resize to 256×256             │
│                                                          │
│  OUTPUT: 256×256 BGR Face Crop                           │
└──────────────────────────────────────────────────────────┘
```

Figure 4.1 — Haar Cascade face detector block diagram

4.2 ControlNet Lineart Extractor

```
┌──────────────────────────────────────────────────────────┐
│          CONTROLNET LINEART DETECTOR — BLOCK DIAGRAM     │
├──────────────────────────────────────────────────────────┤
│  INPUT: 256×256 RGB Face Crop                            │
│  [1] Encoder Backbone (ResNet-34 pretrained)             │
│       Conv(3→64,7×7,s=2)→BN→ReLU → MaxPool               │
│       ResBlock×3[64]  ResBlock×4[128]  ResBlock×6[256]   │
│  [2] Decoder / Feature Pyramid                           │
│       Upsample×4 → Conv(512→256) → Skip fusion           │
│       Upsample×4 → Conv(256→128) → Skip fusion           │
│       Upsample×2 → Conv(128→64)  → Skip fusion           │
│  [3] Output Head: Conv(64→1,1×1) → Sigmoid → binary mask │
│  [4] Post-processing: Invert → 3-channel RGB             │
│                                                          │
│  OUTPUT: 256×256 RGB Lineart Sketch (black lines, white bg)│
└──────────────────────────────────────────────────────────┘
```

Figure 4.2 — ControlNet LineartDetector architecture

4.3 Pix2Pix Conditional GAN — Layer-by-Layer

Generator (U-Net 256)

```
╔══════════════════════════════════════════════════════════╗
║    PIX2PIX GENERATOR (U-Net 256) — LAYER-BY-LAYER        ║
╠══════════════════════════════════════════════════════════╣
║  INPUT: 256×256×3 Sketch Tensor (normalized to [-1, 1])  ║
╠═══════════════════╦═══════════════╦══════════════════════╣
║  ENCODER (DOWN)   ║  Channels     ║  Spatial Size        ║
╠═══════════════════╬═══════════════╬══════════════════════╣
║  e1: Conv         ║  3 → 64       ║  256 → 128           ║
```

```
╔═══════════════════════╦═══════════════════╦═══════════════════════════╗
  e2: Conv-BN-LReLU     ‖  64 → 128         ‖  128 → 64
  e3: Conv-BN-LReLU     ‖  128 → 256        ‖  64 → 32
  e4-e8: Conv-BN        ‖  256 → 512        ‖  32 → 1 (bottleneck)
╠═══════════════════════╬═══════════════════╬═══════════════════════════╣
  DECODER (UP)          ‖  Ch (+ skip)      ‖  Spatial Size
╠═══════════════════════╬═══════════════════╬═══════════════════════════╣
  d1-d7: TConv          ‖  1024 → 64        ‖  1 → 128  + Skip
  d8: TConv-Tanh        ‖  128 → 3          ‖  128 → 256
╠═══════════════════════╩═══════════════════╩═══════════════════════════╣
  OUTPUT: 256×256×3 Synthesized Face Tensor (range [-1, 1])             ‖
╚═══════════════════════════════════════════════════════════════════════╝
```

Figure 4.3a — Pix2Pix U-Net 256 Generator architecture

Discriminator (PatchGAN 70×70)

```
╔═══════════════════════════════════════════════════════════════════════╗
  PIX2PIX DISCRIMINATOR (PatchGAN 70×70) — LAYER-BY-LAYER               ‖
╠═══════════════════════════════════════════════════════════════════════╣
  INPUT: 256×256×6 (Sketch + Generated/Real Face concatenated)          ‖
  L1: Conv(6→64,    4×4, s=2) → LeakyReLU(0.2)                          ‖
  L2: Conv(64→128,  4×4, s=2) → BatchNorm → LeakyReLU(0.2)              ‖
  L3: Conv(128→256,4×4, s=2) → BatchNorm → LeakyReLU(0.2)               ‖
  L4: Conv(256→512,4×4, s=1) → BatchNorm → LeakyReLU(0.2)               ‖
  L5: Conv(512→1,   4×4, s=1) → Sigmoid                                 ‖
╠═══════════════════════════════════════════════════════════════════════╣
  RECEPTIVE FIELD: 70×70 pixels per output patch                        ‖
  OUTPUT: 30×30×1 Patch Validity Map  (0=Fake, 1=Real)                  ‖
╚═══════════════════════════════════════════════════════════════════════╝
```

Figure 4.3b — PatchGAN Discriminator architecture

Loss functions:

```
L_cGAN(G,D) = E[log D(x,y)] + E[log(1 − D(x, G(x)))]
L_L1(G)     = E[||y − G(x)||₁]
L_total      = L_cGAN + λ·L_L1     where λ = 100
```

Pix2Pix Training Pseudocode

```
INPUT:  Paired dataset D = {(x_i, y_i)}  where x=sketch, y=target face
OUTPUT: Trained Generator G, Discriminator D

HYPERPARAMETERS:
  epochs=200, batch_size=1, lr_G=0.0002, lr_D=0.0002
  beta1=0.5, lambda_L1=100

FOR epoch = 1 TO epochs:
  FOR each (x, y) in DataLoader(D):
    // Update Discriminator
    fake_y   <- G(x)
    loss_D   <- (BCE(D(x,y), 1) + BCE(D(x,fake_y.detach()), 0)) / 2
    backprop(loss_D)

    // Update Generator
    loss_G   <- BCE(D(x,fake_y), 1) + lambda_L1 * L1(fake_y, y)
    backprop(loss_G)

  IF epoch % 50 == 0: save_checkpoint(G, D, epoch)
```

Algorithm 4.1 — Pix2Pix GAN Training Pseudocode

4.4 CodeFormer Transformer — Layer-by-Layer

```
╔═══════════════════════════════════════════════════════════════════════╗
          CODEFORMER ARCHITECTURE — LAYER-BY-LAYER                      ‖
```

```
╔═══════════════════════════════════════════════════════════════╗
║  INPUT: 256×256×3 Degraded GAN Output Face                    ║
║                                                               ║
║  STAGE 1: VQ-GAN ENCODER (Feature Extraction)                 ║
║    Conv(3→128) → ResBlock×2 [AvgPool×3] → 8×8×256             ║
║    Attention at 32×32 and 16×16                               ║
║                                                               ║
║  STAGE 2: CODEBOOK LOOKUP (Vector Quantization)               ║
║    Codebook C ∈ R^{1024 × 256}  (1024 codes, dim=256)         ║
║    k* = argmin_k ||z − c_k||₂   (nearest code lookup)         ║
║                                                               ║
║  STAGE 3: TRANSFORMER (Code Prediction)                       ║
║    TransformerBlock × 9: MultiHeadSelfAttention(heads=8,dim=256)
║    Linear(256→1024) → Softmax → Code Index Prediction         ║
║                                                               ║
║  STAGE 4: FIDELITY CONTROL                                    ║
║    z_final = w × z_pred + (1−w) × z_q                         ║
║    w=0.0 → maximum restoration   w=1.0 → preserve GAN output  ║
║                                                               ║
║  STAGE 5: VQ-GAN DECODER  → 256×256×3 Restored Face           ║
╚═══════════════════════════════════════════════════════════════╝
```

Figure 4.4 — CodeFormer layer-by-layer architecture

4.5 SSIM Consistency Verification Module

```
┌─────────────────────────────────────────────────────────────┐
│          SSIM CONSISTENCY VERIFIER — BLOCK DIAGRAM          │
├─────────────────────────────────────────────────────────────┤
│  INPUT A: Original Input Sketch (256×256 grayscale)         │
│  INPUT B: Final Restored Face   (256×256 RGB)               │
│                                                             │
│  [1] Re-Sketch Generated Face via LineartDetector           │
│  [2] Convert both to grayscale                              │
│  [3] SSIM(x,y) = [l(x,y)]^α · [c(x,y)]^β · [s(x,y)]^γ       │
│      score = round(ssim × 100, 1)  → value in [0, 100]%     │
│  [4] Verdict Classification                                 │
│      score >= 70% → 'High Consistency'                      │
│      score >= 50% → 'Moderate Consistency'                  │
│      score  < 50% → 'Low Consistency'                       │
│                                                             │
│  OUTPUT: score (float), verdict (string), re_sketch (image) │
└─────────────────────────────────────────────────────────────┘
```

Figure 4.5 — SSIM Consistency Verification module

4.6 End-to-End System Pseudocode

```
INPUT:  image_path, fidelity_weight w ∈ [0,1]
OUTPUT: pix2pix_face, restored_face, ssim_score, verdict

// STAGE 1: Input Detection & Preprocessing
IF input_type == 'photo':
    bgr_256  <- cv2.resize(crop_face(image, 0.4), (256,256))
    sketch   <- make_sketch(bgr_256)
ELSE:
    sketch   <- cv2.resize(cv2.imread(image_path), (256,256))

// STAGE 2: Pix2Pix GAN Inference
save_pix2pix_input(sketch, temp_dir)
subprocess.run([python, 'test.py', '--model', 'pix2pix', ...])
pix2pix_face <- Image.open(results_dir / stem + '_fake_B.png')

// STAGE 3: CodeFormer Enhancement
subprocess.run([python, 'inference_codeformer.py', '--fidelity_weight', w])
```

```
restored_face <- Image.open(cf_output / 'final_results' / stem + '.png')

// STAGE 4: SSIM Verification
re_sketch  <- make_sketch(restored_face)
ssim_score <- ssim(sketch_gray, re_sketch_gray) * 100
verdict    <- classify_ssim(ssim_score)

RETURN pix2pix_face, restored_face, ssim_score, verdict
```

Algorithm 4.2 — Sketch2Face complete inference pseudocode

5. Hyperparameters

5.1 Pix2Pix GAN Hyperparameters

Hyperparameter	Value	Justification
Batch Size	1	Memory-efficient single-sample adversarial training (standard for Pix2Pix)
Generator LR	0.0002	Adam learning rate; standard for GAN stability
Discriminator LR	0.0002	Same as generator to maintain adversarial balance
Adam beta1	0.5	Reduced momentum for stable GAN gradient updates
Adam beta2	0.999	Standard second moment estimate
Lambda L1	100	High L1 weight encourages pixel-level accuracy alongside adversarial loss
Training Epochs	200	Sufficient for convergence; visualized every 50 epochs
LR Decay Start	100	Linear learning rate decay begins at epoch 100
Image Size (Train)	286×286	Random crop to 256×256 for augmentation during training
Image Size (Test)	256×256	No crop; full resolution inference
Generator Architecture	unet_256	Skip connections preserve spatial features
Discriminator Architecture	PatchGAN (70×70)	Local realism focus captures texture details
Normalization	BatchNorm	Applied in both generator and discriminator

Table 5.1 — Pix2Pix GAN training hyperparameters

5.2 CodeFormer Inference Parameters

Parameter	Value	Description
Fidelity Weight (w)	0.0–1.0 (default: 0.7)	Controls restoration vs. fidelity trade-off
Face Upsample	Enabled	ESRGAN-based background upsampling alongside CodeFormer
Background Enhance	Optional	Enhances non-face regions using Real-ESRGAN
Face Detection	RetinaFace	Built-in CodeFormer face detection for enhancement
Codebook Size	1024 codes × 256 dim	VQ-GAN discrete latent space

Table 5.2 — CodeFormer inference parameters

5.3 Hardware Configuration

Component	Specification
GPU	NVIDIA RTX 5880 Ada Generation (VRAM: 16 GB)
CUDA Version	CUDA 12.x (compute capability sm_89)
RAM	64 GB DDR5
Python	3.11.x
PyTorch	2.x (CUDA-enabled build)
OS	Windows 11 / Ubuntu 22.04 LTS
Training Time	~18 hours (200 epochs, 24,000 pairs, batch=1)

Table 5.3 — Hardware and software environment

6. Training Setup

6.1 Data Pipeline

The PyTorch DataLoader was configured to load paired 512×256 images (sketch|face concatenation) from disk. During training, images were resized to 286×286 and then randomly cropped to 256×256 with random horizontal flipping (augmentation). During validation, images were directly resized to 256×256 without cropping or flipping.

6.2 Training Procedure

Training followed the standard Pix2Pix alternating update procedure:

- Discriminator Update (per batch): Generate fake face → Score real pair and fake pair → Compute combined real/fake BCE loss → Backpropagate and update D weights.
- Generator Update (per batch): Generate fake face → Score through updated D → Compute adversarial loss + λ·L1 → Backpropagate and update G weights.
- Loss Logging: Track D_real, D_fake, G_GAN, G_L1 losses every 100 iterations for convergence monitoring.
- Checkpoint Saving: Generator and discriminator weights saved every 50 epochs for resumable training.
- Learning Rate Decay: Linear decay of both G and D learning rates from epoch 100 to 200 (0.0002 → 0.0).

6.3 Training Commands

```
python train.py \
    --dataroot    ./datasets/sketch2face \
    --name        sketch2face_pix2pix    \
    --model       pix2pix                \
    --direction   AtoB                   \
    --netG        unet_256               \
    --norm        batch                  \
    --n_epochs    100                    \
    --n_epochs_decay 100                 \
    --save_epoch_freq 50                 \
    --batch_size  1                      \
    --gpu_ids     0
```

Command 6.1 — Pix2Pix training launch command

6.4 Inference Pipeline Invocation

```
# Stage 1: Pix2Pix Inference
python test.py \
    --dataroot ./temp/input  --name sketch2face_pix2pix \
    --model pix2pix  --direction AtoB \
    --load_size 256  --crop_size 256  --preprocess none \
    --epoch latest  --eval  --num_test 100

# Stage 2: CodeFormer Enhancement
python inference_codeformer.py \
    --input_path ./cf_input  --output_path ./cf_output \
    --fidelity_weight 0.7  --face_upsample
```

Command 6.2 — Inference pipeline commands (Stage 1 and Stage 2)

7. Results

7.1 Quantitative Results

Metric	Value	Notes
SSIM Consistency (High)	≥ 70%	Reconstruction closely matches input sketch structure
SSIM Consistency (Moderate)	50–69%	Minor structural deviation, manual review advised
SSIM Consistency (Low)	< 50%	Significant divergence — re-synthesis recommended
Observed SSIM Range (test)	71%–93%	Across 200 diverse test samples
Avg Inference Time (Pix2Pix)	~0.8 sec	Single-image forward pass on RTX 5880 Ada
Avg Inference Time (CodeFormer)	~1.3 sec	With face_upsample enabled
Total End-to-End Latency	~2.1 sec	Crop + Pix2Pix + CodeFormer combined
L1 Loss (val, final epoch)	~0.038	Normalized pixel-wise reconstruction error
GAN Loss (generator, final)	~1.2	Adversarial cross-entropy

Table 7.1 — Quantitative performance metrics

7.2 Qualitative Observations

Facial Structure Preservation: The Pix2Pix GAN reliably reproduces gross facial geometry — eye placement, nose bridge width, chin profile, and overall face shape — from lineart input. High SSIM scores (>70%) confirm this.

Texture Generation Quality: Generated faces exhibit realistic skin texture, hair strand direction, and eye highlights. GAN-mode artifacts (checkerboard patterns) are effectively removed by CodeFormer enhancement.

CodeFormer Fidelity Trade-off: At fidelity_weight=0.0, CodeFormer maximally restores detail but may deviate from original sketch geometry. At w=0.7–0.9 (recommended), fine details are sharpened while structure is maintained.

Lineart Quality Sensitivity: System performance correlates strongly with input sketch quality. Clean, well-defined lineart produces SSIM scores consistently above 75%. Noisy or sparse sketches may score below 60%.

7.3 Visual Analytics

The Streamlit application provides three levels of visual output analysis:

- Pipeline Cards — Side-by-side preview of the Sketch, Pix2Pix Output, and CodeFormer Enhanced output with fade-up animations and glow effects on the final result.
- Before/After Comparison Slider — Interactive drag-reveal slider comparing the input sketch against the generated face at full resolution.
- Enhancement Difference Heatmap — Pixel-level absolute difference between Pix2Pix and CodeFormer outputs visualized using the INFERNO colormap, highlighting where CodeFormer made the most corrections.

8. Comparison with Related Work

Sketch2Face is positioned against existing forensic sketch synthesis and face restoration literature. The following table summarizes key differentiating factors:

System / Method	Architecture	Quantitative QA	Blind Restoration	Interactive UI
Sketch2Face (Ours)	Pix2Pix + CodeFormer	SSIM (Yes)	CodeFormer	Streamlit App
Wang & Tang (2009)	MRF Probabilistic	No	No	None
Isola et al. (2017)	Pix2Pix GAN	FID only	No	None
ESRGAN (2021)	Super-Resolution GAN	PSNR/SSIM	Partial	None
Generic Pix2Pix	U-Net 256	FID/PSNR	No	None
StyleGAN2 (Karras)	Progressive GAN	FID/PPL	No	None

Table 8.1 — Comparison of Sketch2Face against related systems

8.1 Key Advantages of Sketch2Face

Forensic-Specific Optimization: Unlike general image-to-image translation models, Sketch2Face was trained exclusively on facial data with a loss function weighting (λ=100 for L1) tuned for pixel-accurate structural reconstruction rather than perceptual plausibility.

Quantitative Identity Verification: No prior forensic sketch synthesis system provides an automated, mathematically grounded identity preservation score. The SSIM consistency module fills this critical gap.

Integrated Two-Stage Pipeline: The combination of Pix2Pix synthesis with CodeFormer restoration in a single automated pipeline represents a novel contribution — GAN artifacts that would otherwise compromise forensic utility are systematically eliminated by the transformer restoration stage.

Accessible Deployment: The Streamlit web interface makes the system accessible to forensic investigators without machine learning expertise, removing the technical barrier to practical forensic deployment.

9. Limitations & Challenges

9.1 Technical Limitations

Fixed 256×256 Resolution: The core GAN was trained and operates at 256×256 pixels, which is insufficient for high-fidelity forensic identification in real scenarios where facial minutiae are required. While CodeFormer provides perceptual upsampling, the base feature resolution constrains identity-defining detail.

Frontal Face Bias: The CelebA-HQ training corpus is heavily weighted toward frontal, well-lit facial portraits. The system performs poorly on non-frontal poses (profile, three-quarter view), extreme lighting conditions, or occluded faces — all of which are common in real forensic scenarios.

Sketch Style Dependency: Performance correlates strongly with input sketch quality. The Lineart extractor generates normalized, consistent sketches from photos, but hand-drawn sketches submitted directly vary unpredictably in style, which the GAN may fail to generalize across effectively.

Computational Infrastructure: A CUDA-capable GPU with ≥ 8 GB VRAM is required for practical inference speeds (~2.1 seconds). CPU-only inference is technically possible but prohibitively slow for operational forensic use.

Demographic Bias Inheritance: CelebA-HQ over-represents certain demographic groups. This inherited bias may result in lower reconstruction fidelity for underrepresented ethnicities and age groups.

9.2 Encountered Challenges

GAN Training Instability: Early training runs suffered from mode collapse where the generator produced blurry, averaged faces regardless of input. Resolved by enforcing batch_size=1 and carefully tuning the λ=100 L1 weight.

CORS Restrictions on Local Assets: The Streamlit application required special handling for locally served assets to avoid browser CORS policy blocking, resolved via the integrated HTTP server in launcher.py.

CodeFormer Output Path Resolution: CodeFormer generates output filenames with modified suffixes. Robust filename inference logic was implemented to locate the correct enhanced output file regardless of filename format variations.

10. Future Work

Several clear directions exist for extending Sketch2Face toward a production-grade forensic synthesis system:

1. High-Resolution Synthesis (512×512 / 1024×1024): Upgrade the base GAN using progressive growing techniques or integrate a latent diffusion model backbone to natively generate at higher resolutions, significantly improving fine-grained facial detail for operational forensic use.

2. Diffusion Model Integration: Replace or augment the Pix2Pix GAN with a ControlNet-conditioned Stable Diffusion model (Zhang et al., 2023). Diffusion models offer superior texture diversity, more realistic hair and skin rendering, and better handling of diverse sketch styles.

3. Multi-Pose Sketch Support: Extend training data collection to include profile and three-quarter view face-sketch pairs, and modify the architecture to handle 3D pose estimation alongside 2D synthesis.

4. Mobile & Edge Deployment: Quantize the Pix2Pix and CodeFormer models to INT8 precision using TensorRT and deploy on mobile GPU platforms for field operations without requiring a server connection.

5. Interactive Real-Time Sketch Canvas: Integrate a browser-based drawing canvas (using fabric.js or p5.js) that feeds directly into the synthesis pipeline with real-time preview updates as investigators refine the sketch.

6. Cross-Domain Sketch Adaptation: Develop domain adaptation modules that handle diverse sketch styles (pencil, charcoal, composite digital tools like FACES or IdentiKit) using style-invariant feature extraction.

7. Demographic Bias Correction: Curate additional training data from BUPT-Balanced Face and other demographically balanced datasets to correct ethnic and age group under-representation in reconstruction quality.

8. Face Video Synthesis: Extend the system to generate animated portrait videos from sketch descriptions using video prediction networks, enabling richer forensic evidence for witness verification.

> **Research Vision: The ultimate goal is to develop Sketch2Face into a certified forensic tool that integrates directly with national face recognition databases, providing end-to-end AI-assisted suspect identification from witness-described sketches with full evidentiary auditability.**